

Analyzing data on the level of cognitive processes - ESCoP 2025

Gidon T. Frischkorn, Ruhi Bhanap, and Isabel Courage

Exercise 6: Full Diffusion Decision Model

In this exercise, we will have a look at estimating the parameters from the diffusion decision model using the hierarchical Bayesian implementation of the full Diffusion Decision Model.

Your task is to load one of the data sets we prepared for this exercise, `exercercise5_visual_search_task.txt` or `exercercise5_color_judgement_task.txt`.

The Visual Search data set contains data for 33 participants that completed the Visual Search task. In this task, participants were presented a color cue, a positive (the target color), a negative (the non target color) or neutral (not part of the visual search display). Participants had to look for a “C” that either looked upwards or downwards and report it’s direction with the arrow key.

In this exercise, we implement the four parameter `ddm` and test if participants were biased in responding with one of the two response options (i.e. up or down). For this we have to specify a `ddm` with the actual responses - that is up or down - as the response boundaries. We will estimate the `drift` separately for the correct boundry. The correct variable is part of this dataset.

Solution

```
# Load required packages
library(bmm)
library(tidyverse)
library(here)
```

1) prepare the data

First, we need to prepare the data so that we can analyze our data with the `ddm` using the `bmm` function. For this we need to include a numeric `response` variable coding arrowup as 0 and arrowdown as 1. Convert the reaction times to seconds,

```
ddm_data <- read.table(here("data", "exercercise5_visual_search_task.txt"),
                      header = T)

ddm_data$rt <- ddm_data$RT/1000
ddm_data$resp_num <- ifelse(ddm_data$response == "arrowdown", 1, 0)
ddm_data$condition <- factor(ddm_data$condition,
                             levels = c("negative", "neutral", "positive"))
```

2) Specify the ddm model object

Next, we need to specify the model object to link which data columns are used to identify the `ddm` using the `rt` in seconds and the newly coded `response` variable as the required variables

```
ddm_model <- ddm(rt = "rt", response = "resp_num", version = "four_par")
```

3) specify the bmmformula with

Now, we can consider which parameters are affected by the experimental manipulations. Here we assume different `drift` and `ndt` dependent on the the type of cue (positive/negative/neutral) and a `drift` based on the correct response. Apart from that I only include fixed and random intercepts for the boundary and relative starting point.

```
ddm_formula <- bmf(
  drift ~ 0 + correct:condition + (0 + correct:condition || participant),
  bound ~ 1 + (1 || participant),
  ndt ~ 0 + condition + (0 + condition || participant),
  zr ~ 1 + (1 || participant)
)

default_prior(ddm_formula, data = ddm_data, model = ddm_model)
```

##	prior	class	coef	group
##	student_t(3, 0, 2.5)	sd		
##	student_t(3, 0, 2.5)	sd		participant
##	student_t(3, 0, 2.5)	sd	Intercept	participant
##	cauchy(0,1)	b	correctarrowdown:conditionnegative	
##	cauchy(0,1)	b	correctarrowdown:conditionneutral	
##	cauchy(0,1)	b	correctarrowdown:conditionpositive	
##	cauchy(0,1)	b	correctarrowup:conditionnegative	
##	cauchy(0,1)	b	correctarrowup:conditionneutral	
##	cauchy(0,1)	b	correctarrowup:conditionpositive	
##	student_t(3, 0, 2.5)	sd		
##	student_t(3, 0, 2.5)	sd		participant
##	student_t(3, 0, 2.5)	sd	correctarrowdown:conditionnegative	participant
##	student_t(3, 0, 2.5)	sd	correctarrowdown:conditionneutral	participant
##	student_t(3, 0, 2.5)	sd	correctarrowdown:conditionpositive	participant
##	student_t(3, 0, 2.5)	sd	correctarrowup:conditionnegative	participant
##	student_t(3, 0, 2.5)	sd	correctarrowup:conditionneutral	participant
##	student_t(3, 0, 2.5)	sd	correctarrowup:conditionpositive	participant
##	normal(-1.5,0.5)	b	conditionnegative	
##	normal(-1.5,0.5)	b	conditionneutral	
##	normal(-1.5,0.5)	b	conditionpositive	
##	student_t(3, 0, 2.5)	sd		
##	student_t(3, 0, 2.5)	sd		participant
##	student_t(3, 0, 2.5)	sd	conditionnegative	participant
##	student_t(3, 0, 2.5)	sd	conditionneutral	participant
##	student_t(3, 0, 2.5)	sd	conditionpositive	participant
##	student_t(3, 0, 2.5)	sd		
##	student_t(3, 0, 2.5)	sd		participant
##	student_t(3, 0, 2.5)	sd	Intercept	participant
##	cauchy(0,1)	b		
##	normal(0,0.5)	Intercept		
##	normal(-1.5,0.5)	b		
##	normal(0,0.5)	Intercept		
##	constant(0)	Intercept		
##	constant(0)	Intercept		
##	constant(0)	Intercept		
##	constant(0)	Intercept		
##	resp dpar nlpar lb ub		source	
##	bound	0	default	

```

##      bound      0      (vectorized)
##      bound      0      (vectorized)
##      drift    <NA> <NA> (vectorized)
##      drift    <NA> <NA> (vectorized)
##      drift    <NA> <NA> (vectorized)
##      drift    <NA> <NA> (vectorized)
##      drift    <NA> <NA> (vectorized)
##      drift    <NA> <NA> (vectorized)
##      drift      0      default
##      drift      0      (vectorized)
##      drift      0      (vectorized)
##      drift      0      (vectorized)
##      drift      0      (vectorized)
##      drift      0      (vectorized)
##      drift      0      (vectorized)
##      drift      0      (vectorized)
##      ndt      <NA> <NA> (vectorized)
##      ndt      <NA> <NA> (vectorized)
##      ndt      <NA> <NA> (vectorized)
##      ndt      0      default
##      ndt      0      (vectorized)
##      ndt      0      (vectorized)
##      ndt      0      (vectorized)
##      ndt      0      (vectorized)
##      zr      0      default
##      zr      0      (vectorized)
##      zr      0      (vectorized)
##      drift    <NA> <NA>      user
##      bound    <NA> <NA>      user
##      ndt      <NA> <NA>      user
##      zr      <NA> <NA>      user
##      <NA> <NA>      user
##      sdrift    <NA> <NA>      user
##      sndt      <NA> <NA>      user
##      szr      <NA> <NA>      user

```

4) estimate the model

Finally, we use the `bmm` function to fit the full DDM by passing the `ddm` model object and the `bmmformula`. Again, I saved the model results to save time. The `ddm` can take quite some time for parameter estimation. For this data it took about an hour on my machine.

```

ddm_fit <- bmm(
  formula = ddm_formula,
  data = ddm_data,
  model = ddm_model,
  cores = 4,
  iter = 2000,
  refresh = 0,
  sample_prior = TRUE,
  backend = "cmdstanr",
  file = here("models", "ddm_visualesearch_exc6")
)

```

5) Evaluate results

First, we can have a look at the summary of the fitted `ddm` model object.

```
summary(ddm_fit)
```

```
## Loading required package: rstan
## Warning: package 'rstan' was built under R version 4.4.3
## Loading required package: StanHeaders
## Warning: package 'StanHeaders' was built under R version 4.4.3
##
## rstan version 2.36.0.9000 (Stan version 2.37.0)
## For execution on a local, multicore CPU with excess RAM we recommend calling
## options(mc.cores = parallel::detectCores()).
## To avoid recompilation of unchanged Stan programs, we recommend calling
## rstan_options(auto_write = TRUE)
## For within-chain threading using `reduce_sum()` or `map_rect()` Stan functions,
## change `threads_per_chain` option:
## rstan_options(threads_per_chain = 1)
##
## Attaching package: 'rstan'
## The following object is masked from 'package:tidyr':
##
##      extract
##
## Model: ddm(rt = "rt",
##           response = "resp_num",
##           version = "four_par")
## Links: drift = identity; bound = log; ndt = log; zr = logit; sdrift = identity; sndt = identity; szr = identity
## Formula: drift ~ 0 + correct:condition + (0 + correct:condition || participant)
##           bound ~ 1 + (1 || participant)
##           ndt ~ 0 + condition + (0 + condition || participant)
##           zr ~ 1 + (1 || participant)
##           sdrift = 0
##           sndt = 0
##           szr = 0
## Data: ddm_data (Number of observations: 5388)
## Draws: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;
##           total post-warmup draws = 4000
##
## Multilevel Hyperparameters:
## ~participant (Number of levels: 33)
##
##           Estimate Est.Error 1-95% CI
## sd(drift_correctarrowdown:conditionnegative)    0.07    0.05    0.00
## sd(drift_correctarrowup:conditionnegative)       0.07    0.05    0.00
## sd(drift_correctarrowdown:conditionneutral)      0.05    0.04    0.00
## sd(drift_correctarrowup:conditionneutral)        0.05    0.04    0.00
## sd(drift_correctarrowdown:conditionpositive)     0.15    0.06    0.03
## sd(drift_correctarrowup:conditionpositive)       0.18    0.06    0.07
## sd(bound_Intercept)                            0.06    0.02    0.02
## sd(ndt_conditionnegative)                       0.33    0.05    0.25
## sd(ndt_conditionneutral)                        0.43    0.07    0.31
```

```
## sd(ndt_conditionpositive)          0.40      0.06      0.30
## sd(zr_Intercept)                   0.05      0.02      0.01
##                                u-95% CI Rhat Bulk_ESS Tail_ESS
## sd(drift_correctarrowdown:conditionnegative) 0.18 1.00    1340    2336
## sd(drift_correctarrowup:conditionnegative)    0.18 1.00    1269    1767
## sd(drift_correctarrowdown:conditionneutral)   0.14 1.00    1736    2029
## sd(drift_correctarrowup:conditionneutral)     0.14 1.00    1624    2151
## sd(drift_correctarrowdown:conditionpositive)  0.27 1.00     993    1061
## sd(drift_correctarrowup:conditionpositive)    0.30 1.00     979    1242
## sd(bound_Intercept)                       0.10 1.00    1073    1280
## sd(ndt_conditionnegative)                   0.43 1.00    1513    2357
## sd(ndt_conditionneutral)                    0.59 1.00    1442    1820
## sd(ndt_conditionpositive)                   0.55 1.00    1539    2118
## sd(zr_Intercept)                           0.10 1.00     960    1199
##
## Regression Coefficients:
##                                Estimate Est.Error l-95% CI u-95% CI
## bound_Intercept                1.31      0.02      1.27      1.36
## zr_Intercept                   -0.04      0.02     -0.09      0.00
## drift_correctarrowdown:conditionnegative    1.60      0.04      1.52      1.69
## drift_correctarrowup:conditionnegative     -1.58      0.04     -1.66     -1.50
## drift_correctarrowdown:conditionneutral     1.38      0.04      1.30      1.46
## drift_correctarrowup:conditionneutral      -1.27      0.04     -1.35     -1.20
## drift_correctarrowdown:conditionpositive    1.76      0.05      1.66      1.86
## drift_correctarrowup:conditionpositive     -1.61      0.05     -1.72     -1.51
## ndt_conditionnegative              -0.80      0.07     -0.94     -0.68
## ndt_conditionneutral               -1.02      0.09     -1.20     -0.85
## ndt_conditionpositive              -1.03      0.08     -1.20     -0.88
##                                Rhat Bulk_ESS Tail_ESS
## bound_Intercept                1.00    2419    2599
## zr_Intercept                   1.00    3582    3066
## drift_correctarrowdown:conditionnegative  1.00    3582    3056
## drift_correctarrowup:conditionnegative    1.00    3462    2866
## drift_correctarrowdown:conditionneutral   1.00    4063    3550
## drift_correctarrowup:conditionneutral     1.00    4301    3297
## drift_correctarrowdown:conditionpositive  1.00    2944    2893
## drift_correctarrowup:conditionpositive    1.00    2828    2338
## ndt_conditionnegative              1.00    1159    1683
## ndt_conditionneutral               1.00    1021    1510
## ndt_conditionpositive              1.00    1109    2027
##
## Constant Parameters:
##                                Value
## sdrift_Intercept                0.00
## sndt_Intercept                  0.00
## szr_Intercept                   0.00
##
## Draws were sampled using sample(hmc). For each parameter, Bulk_ESS
## and Tail_ESS are effective sample size measures, and Rhat is the potential
## scale reduction factor on split chains (at convergence, Rhat = 1).
```

Then, we can test some hypothesis:

```
ddm_hypothesis <- c(
  # Drift rates: comparing arrow down vs arrow up within each condition
```

```

drift_neg = "drift_correctarrowdown:conditionnegative = -drift_correctarrowup:conditionnegative",
drift_neu = "drift_correctarrowdown:conditionneutral = -drift_correctarrowup:conditionneutral",
drift_pos = "drift_correctarrowdown:conditionpositive = -drift_correctarrowup:conditionpositive",

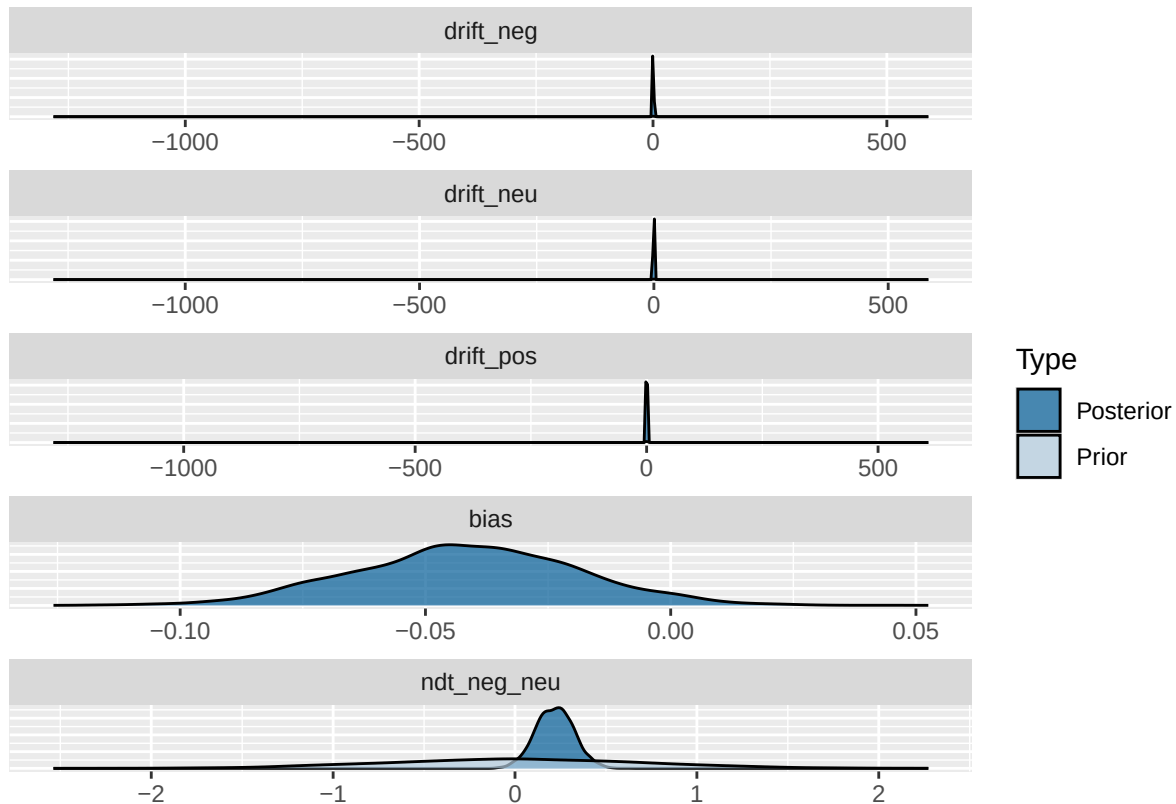
# Starting point / bias
bias = "zr_Intercept = 0",

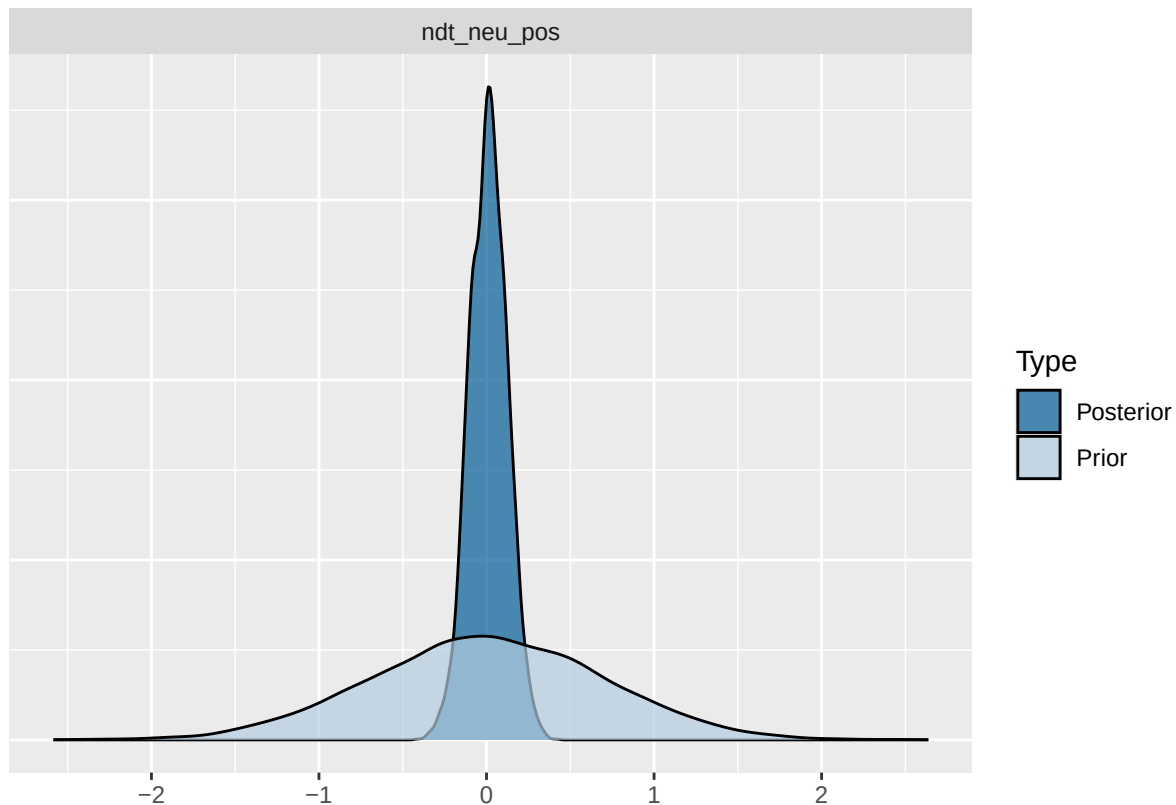
# Non-decision time comparisons
ndt_neg_neu = "ndt_conditionnegative = ndt_conditionneutral",
ndt_neu_pos = "ndt_conditionneutral = ndt_conditionpositive"
)

ddm_hypothesis_results <- brms::hypothesis(
  ddm_fit,
  ddm_hypothesis
)

plot(ddm_hypothesis_results)

```





```
ddm_hypothesis_results
```

```
## Hypothesis Tests for class b:
```

##	Hypothesis	Estimate	Est.Error	CI.Lower	CI.Upper	Evid.Ratio	Post.Prob	Star
## 1	drift_neg	0.02	0.06	-0.09	0.14	42.00	0.98	
## 2	drift_neu	0.11	0.05	0.00	0.21	6.76	0.87	*
## 3	drift_pos	0.15	0.07	0.00	0.29	5.16	0.84	
## 4	bias	-0.04	0.02	-0.09	0.00	NA	NA	
## 5	ndt_neg_neu	0.22	0.10	0.02	0.42	0.81	0.45	*
## 6	ndt_neu_pos	0.01	0.11	-0.21	0.23	6.18	0.86	

```
## ---
```

```
## 'CI': 90%-CI for one-sided and 95%-CI for two-sided hypotheses.
```

```
## '*': For one-sided hypotheses, the posterior probability exceeds 95%;
```

```
## for two-sided hypotheses, the value tested against lies outside the 95%-CI.
```

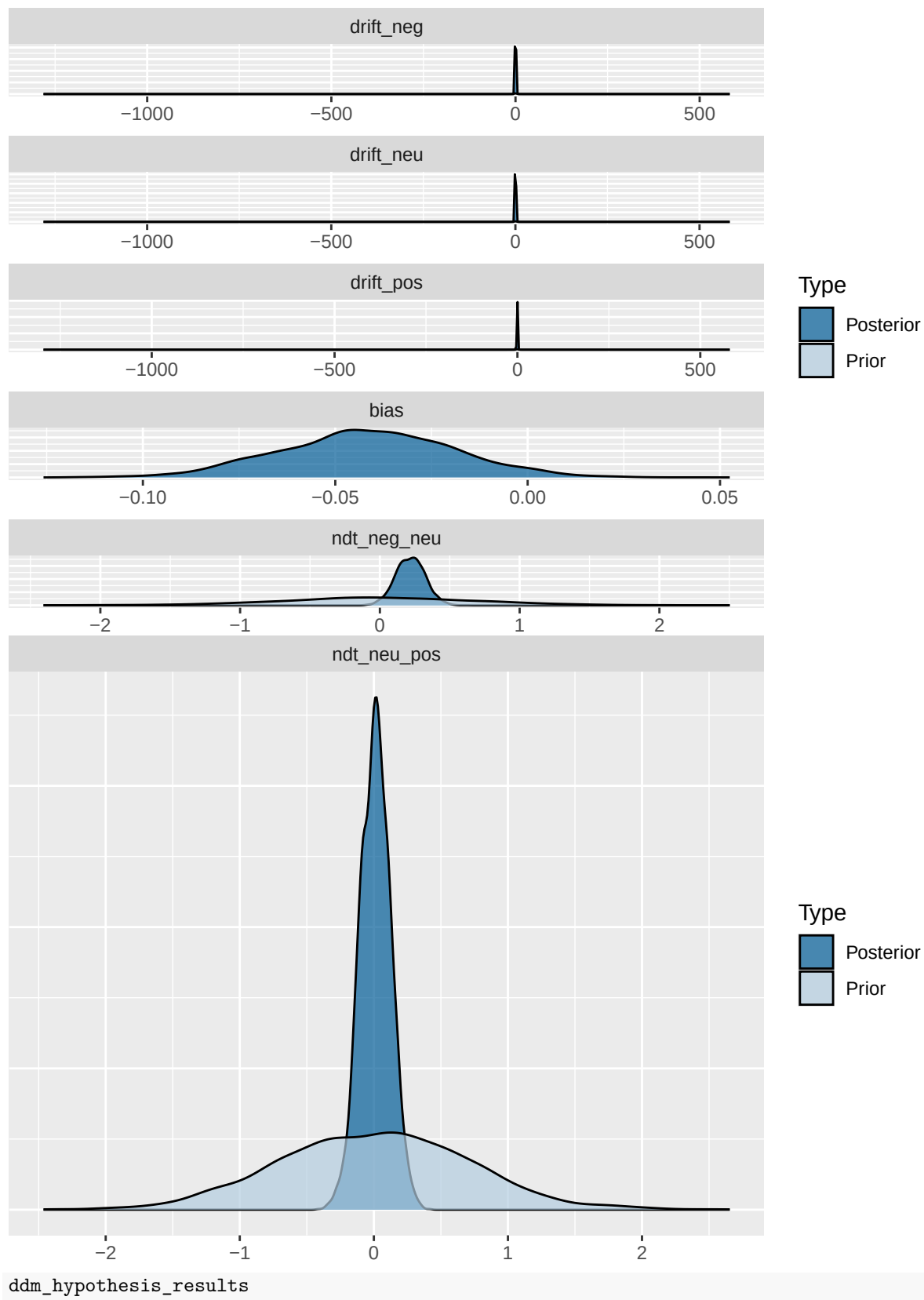
```
## Posterior probabilities of point hypotheses assume equal prior probabilities.
```

```
1/abs(ddm_hypothesis_results$hypothesis$Evid.Ratio)
```

```
## [1] 0.02381142 0.14786518 0.19381960 NA 1.23844366 0.16170928
```

```
ddm_hypothesis_results <- brms::hypothesis(
  ddm_fit,
  ddm_hypothesis
)
```

```
plot(ddm_hypothesis_results)
```



Hypothesis Tests for class b:


```
##      Hypothesis Estimate Est.Error CI.Lower CI.Upper Evid.Ratio Post.Prob Star
## 1  drift_neg      0.02      0.06    -0.09    0.14     41.04      0.98
## 2  drift_neu      0.11      0.05     0.00    0.21      6.98      0.87  *
## 3  drift_pos      0.15      0.07     0.00    0.29      4.90      0.83
## 4      bias     -0.04      0.02    -0.09    0.00        NA        NA
## 5 ndt_neg_neu      0.22      0.10     0.02    0.42      0.78      0.44  *
## 6 ndt_neu_pos      0.01      0.11    -0.21    0.23      6.70      0.87
## ---
## 'CI': 90%-CI for one-sided and 95%-CI for two-sided hypotheses.
## '*': For one-sided hypotheses, the posterior probability exceeds 95%;
## for two-sided hypotheses, the value tested against lies outside the 95%-CI.
## Posterior probabilities of point hypotheses assume equal prior probabilities.
1/abs(ddm_hypothesis_results$hypothesis$Evid.Ratio)

## [1] 0.0243651 0.1431699 0.2038829          NA 1.2861100 0.1493093
```